




# Visión Computacional usando Deep Learning (I)

Ing. Jaime Antonio Huaytalla Pariona
jhuaytalla@uncp.edu.pe

# Contenido

1. **Redes Neuronales**
2. Redes Neuronales Convolucionales (CNN)
3. Transferencia de Aprendizaje
4. Redes Neuronales Recurrentes (RNN)

# Redes Neuronales

## Aplicación en Visión Computacional

- Visión computacional “tradicional”
  - Se basa en extracción manual de atributos (features)
  - Los atributos se usan para el entrenamiento

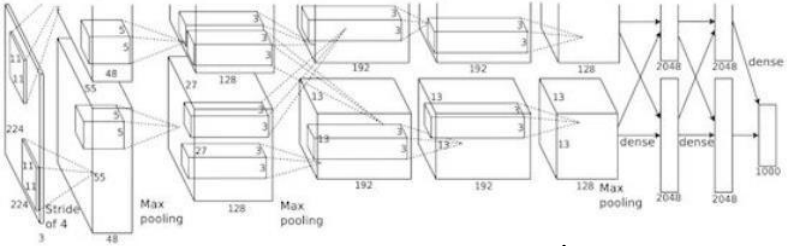





Entrenamiento basado en atributos
(SVM, redes neuronales, random forest, etc.)

- Visión computacional basada en CNN (“Convolutional Neural Networks”)
  - Toda la imagen se usa para el entrenamiento



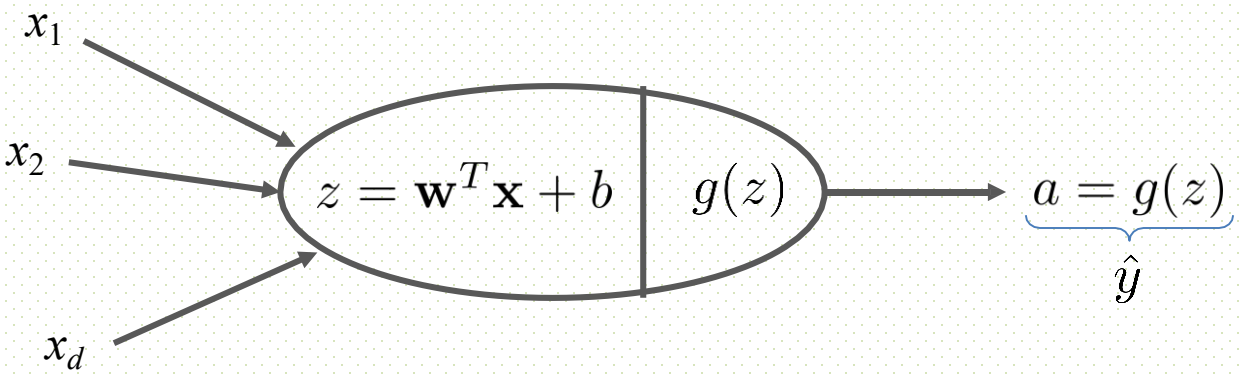



Entrenamiento usando CNN

# Redes Neuronales

## Unidad Neuronal

- Representa una "neurona" dentro de la red (también: unidad logística)




- Partes importantes:
  - *Función de propagación* (combinación lineal de las entradas): $z \in \mathbb{R}$
    - Contiene pesos ($\boldsymbol{w} \in \mathbb{R}^d$) y bias o sesgo ($b \in \mathbb{R}$)
  - *Función de activación*: $g \in \mathbb{R}$

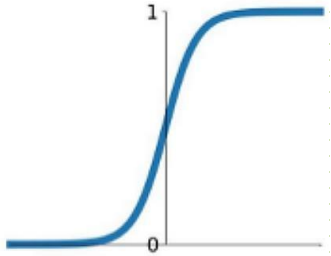
- Entrada: $\boldsymbol{x} \in \mathbb{R}^d$ ($d$ atributos)
- Salida: $\hat{y} \in \mathbb{R}$

# Redes Neuronales

## Funciones de Activación

- Función sigmoidea
  - Mapea a [0,1] (como probabilidad)
  - Se usa para salida de toda la red
  - Problema: “vanishing gradients”

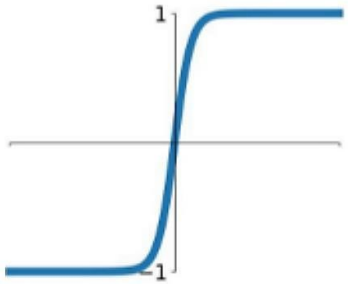
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$




Sigmoidea

- Función tangente hiperbólica
  - Mapea a [-1, 1]
  - Se usa para capas ocultas
  - Mejora el problema de la sigmoidea

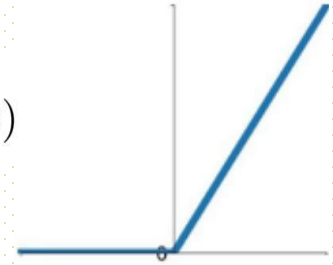
$$a(z) = \tanh(z)$$




Tangente hiperbólica

- Función ReLU (Rectified Linear Unit)
  - Se usa para capas ocultas
  - Ventaja: facilidad de cálculo
  - Problema: genera neuronas “muertas”

$$a(z) = \text{máx}(0, z)$$



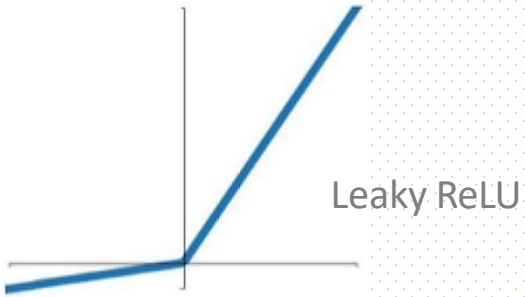

ReLU

# Redes Neuronales

## Funciones de Activación

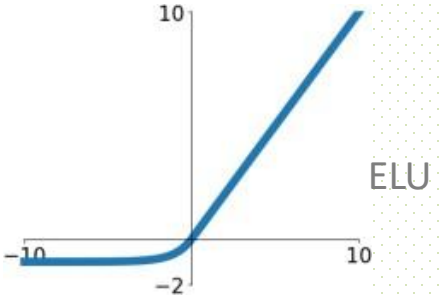
- Función Leaky ReLU
  - Variación de ReLU
  - Se añade una pequeña pendiente a la parte negativa
  - Evita neuronas “muertas” (cero)

$$a(z) = \text{máx}(0.01z, z) = \begin{cases} 0.01z, & \text{si } z < 0 \\ z, & \text{si } z \geq 0 \end{cases}$$




- Función ELU
  - Facilita la diferenciabilidad (función más suave)
  - Problema: computacionalmente costoso

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

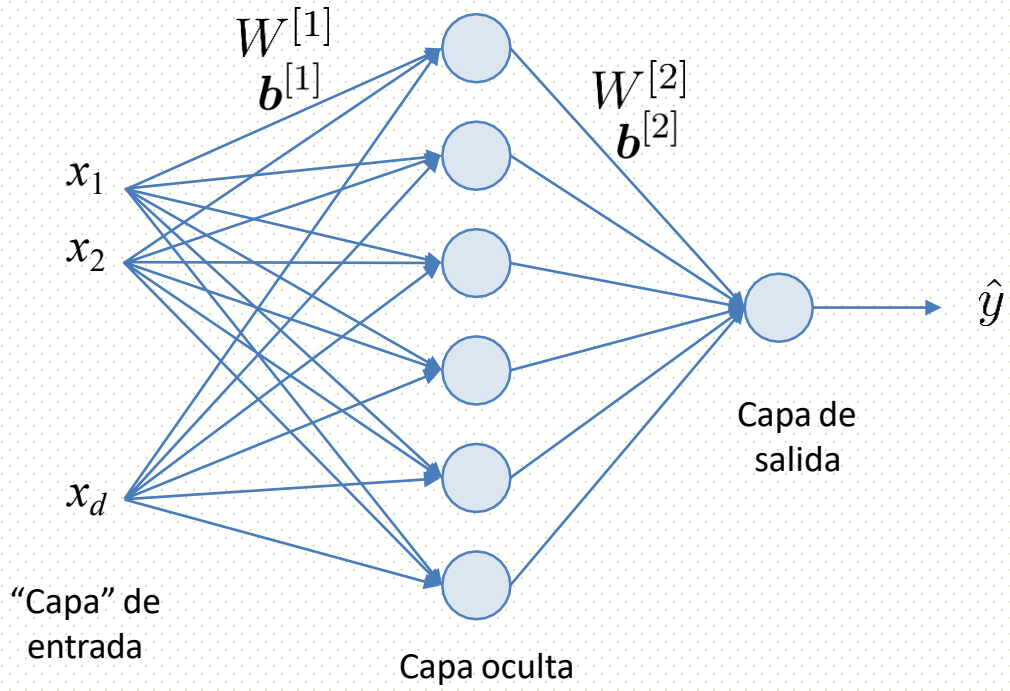



- Otras: Softmax (para varias clases), Maxout, Softplus, Swish, GELU, Mish, etc.

# Redes Neuronales

## Redes Neuronales Simples (pocas capas)

- Ejemplo: red neuronal con una capa oculta y una capa de salida (el corchete indica el número de capa)

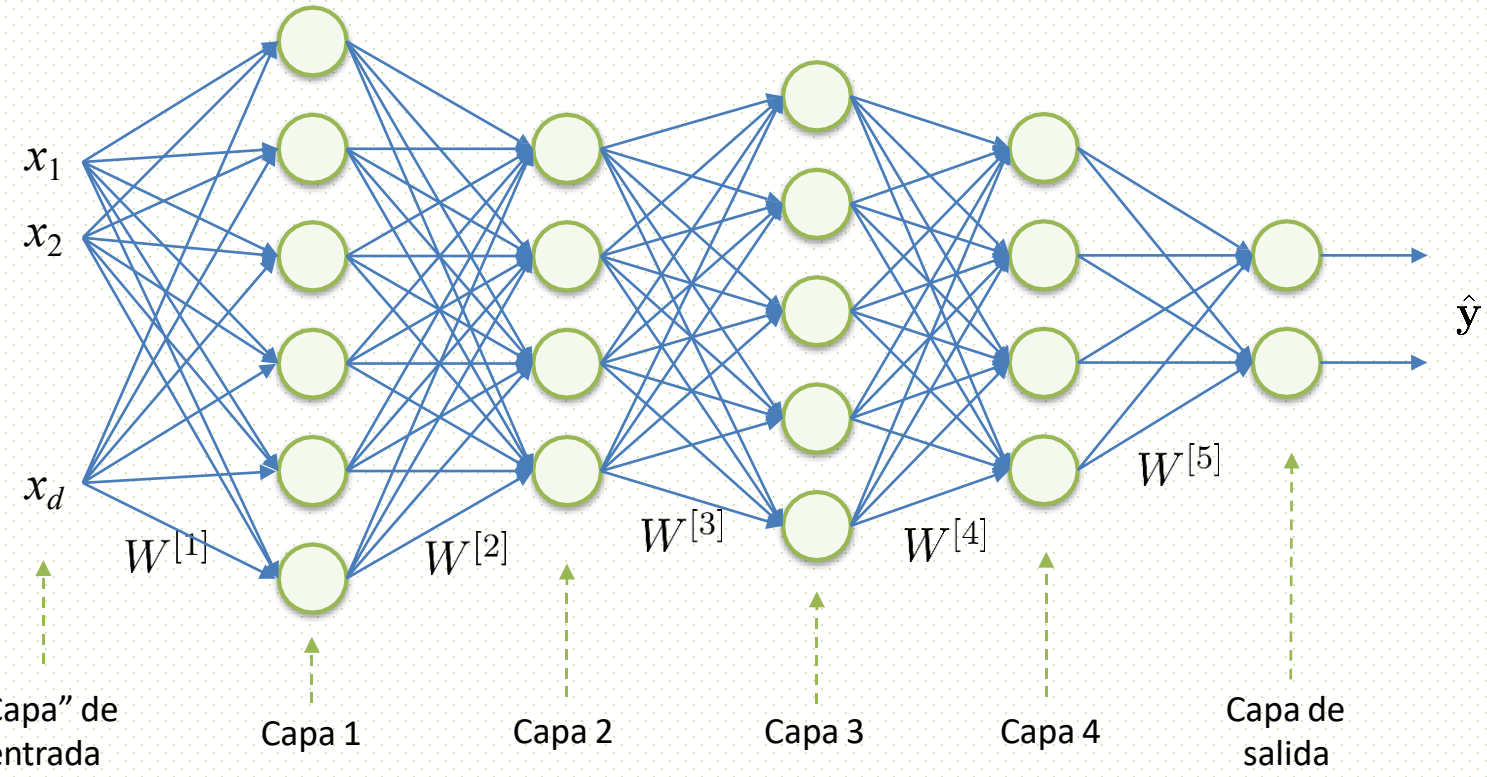



- En general puede haber más de 1 salida (para clasificar varias clases)

# Redes Neuronales

## Redes Neuronales Profundas

- Llamadas: *Deep Neural Networks* (*DNN*)
- Son redes neuronales con varias capas (“fully connected”)



# Redes Neuronales

## Función de Pérdida

- También llamada función de costo o “loss function”
  - Brinda la suma de errores entre la salida correcta ($y_i$) y lo que la red predice ($\hat{y}_i$)
- Para clasificación:
  - *Entropía cruzada Binaria*: se utiliza en problemas de clasificación binarios

$$J(W) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

$N$: total de instancias

  - *Entropía cruzada*: se utiliza en problemas multiclase (con salidas “one-hot encoding”)

$$J(W) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{K}[y_{ij} \log(\hat{y}_{ij})]$$

$K$: total de clases

  - *Entropía cruzada dispersa* (sparse): asigna etiquetas enteras a las clases
- Para regresión:
  - Error cuadrático medio (MSE): $J(W) = \frac{1}{2N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$

# Redes Neuronales

## Función de Pérdida

- Entrenamiento de una red
  - Consiste en obtener pesos ($W$) que minimicen la función de pérdida:

$$\min_W J(W) \quad \Rightarrow \quad \frac{\partial J(W)}{\partial W} = 0$$

  - Se requiere el cálculo de derivadas:
    - Requieren el uso de la regla de la cadena repetidamente (a medida que la red es más grande)
    - Se calculan utilizando "propagación hacia atrás" ("backpropagation")

- Optimización (minimización)
  - Los principales métodos son:
    - Descenso del gradiente: considera todos los datos (resulta lento con muchos datos)
    - Descenso del gradiente estocástico (SGD): considera uno o una muestra de datos
    - SGD + momentum
    - RMSProp
    - Adam, etc.

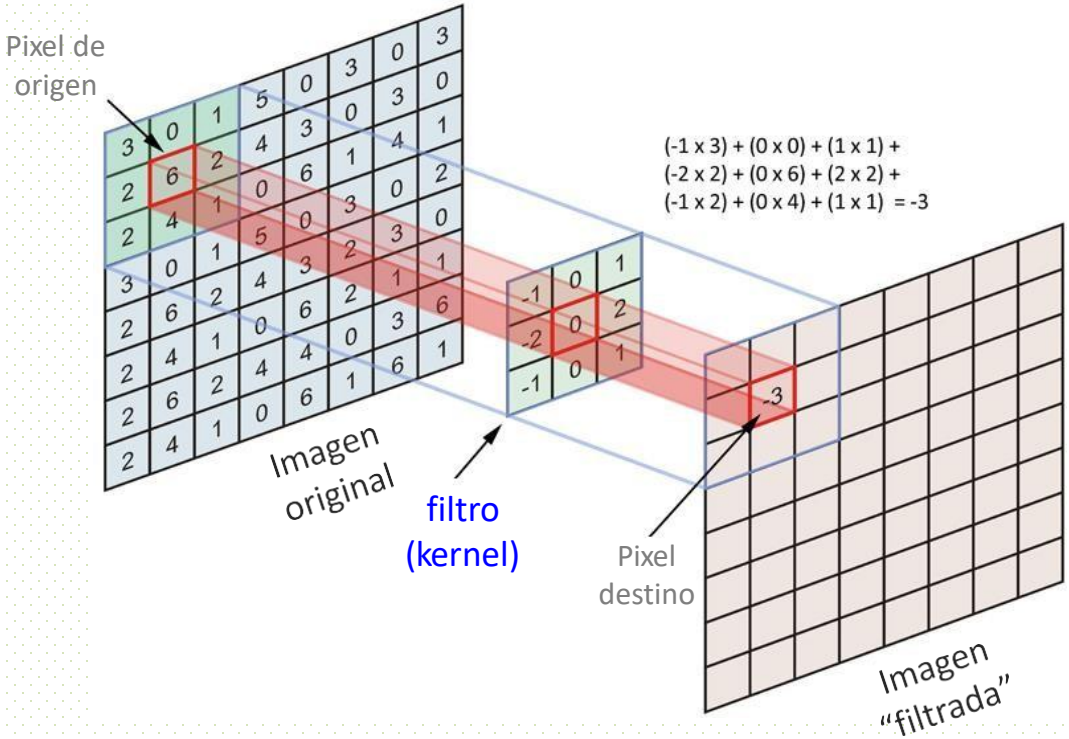
# Contenido

1. Redes Neuronales
2. **Redes Neuronales Convolucionales (CNN)**
3. Transferencia de Aprendizaje
4. Redes Neuronales Recurrentes (RNN)

# Redes Neuronales Convolucionales

## Convoluciones en 2 Dimensiones

- Convolución consiste en:
  - Sobreponer un filtro a la imagen, realizar las multiplicaciones término a término, y sumar el resultado



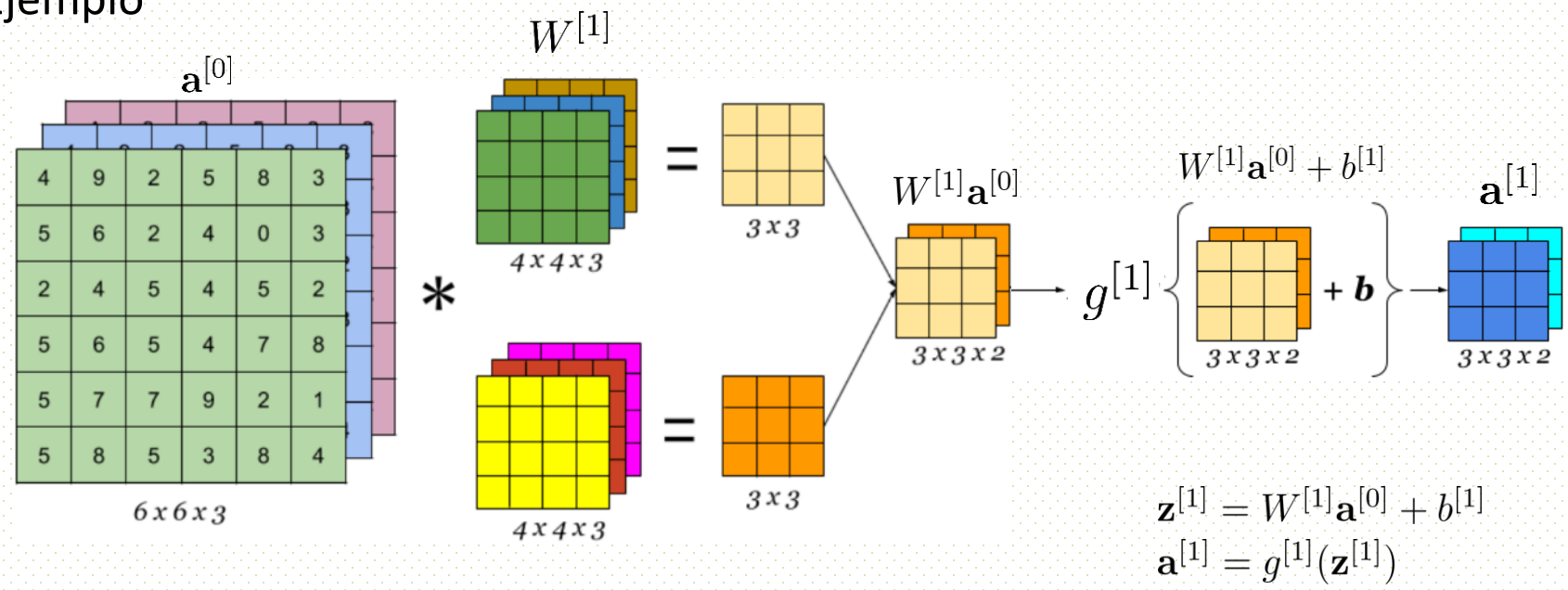

  - El filtro "recorre" toda la imagen

# Redes Neuronales Convolucionales

## Capa de Convolución

- Componentes:
  - Mapa de activación $z$ (convolución de la entrada con los filtros + bias)
  - Funciones de activación $g$ (aplicadas a los mapas de activación)
    - Usualmente ReLU (variaciones de ReLU o tanh)

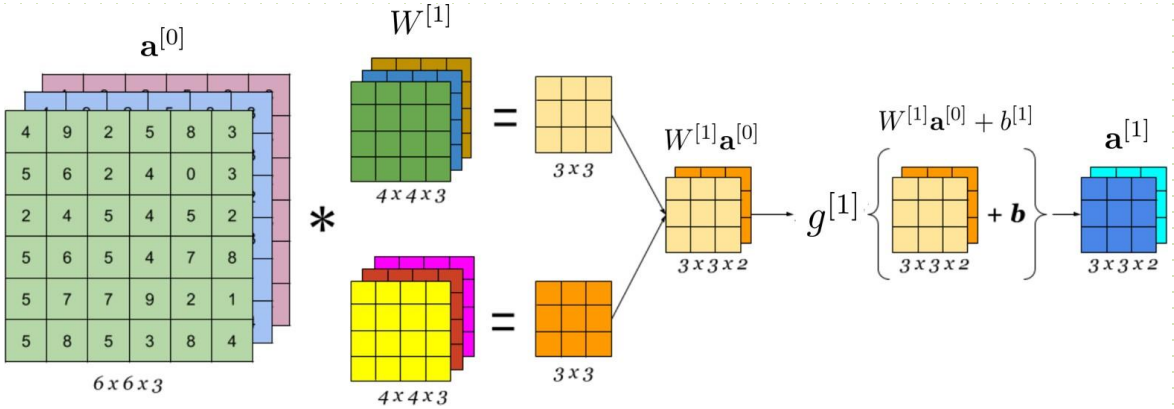
- Ejemplo



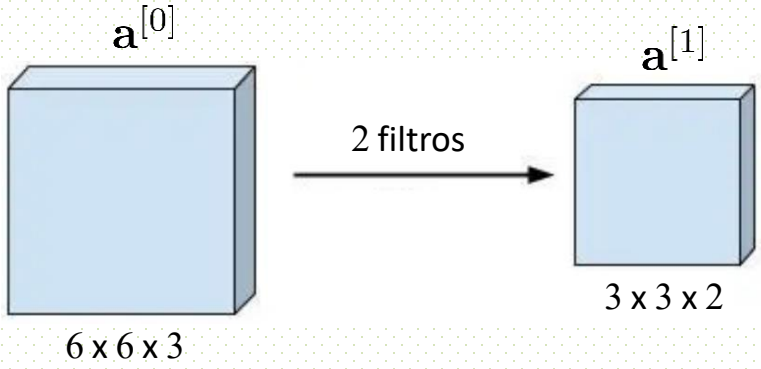
# Redes Neuronales Convolucionales

## Capa de Convolución

- Simplificación de la representación:




- De manera simplificada (cuando hay varias capas):

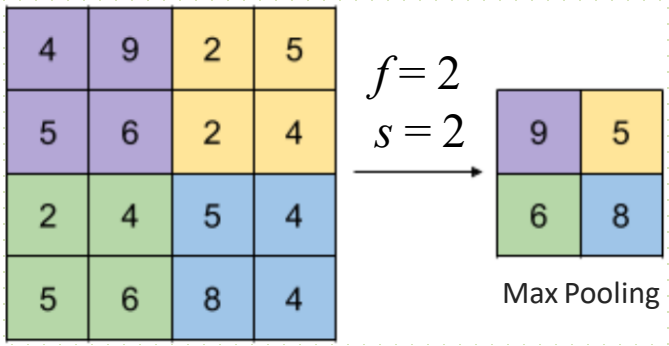


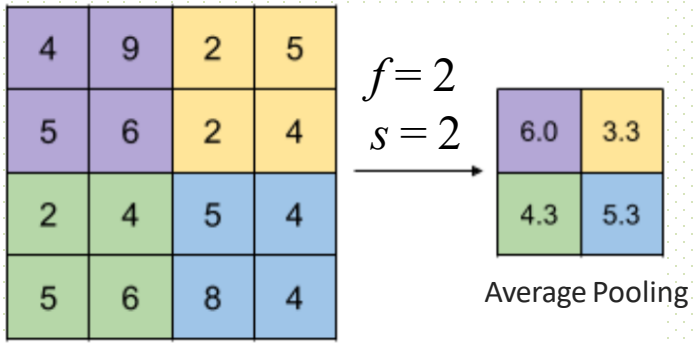
# Redes Neuronales Convolucionales

## Capa de Pooling

- Reduce el tamaño de las representaciones (“downsampling”)
  - Opera en cada canal de manera independiente
  - No introduce ningún nuevo parámetro (no hay que “aprender” nada)

- Tipos más utilizados
  - Max Pooling: utiliza el valor máximo de cada conjunto de píxeles
  - Average Pooling: utiliza el valor promedio de cada conjunto de píxeles

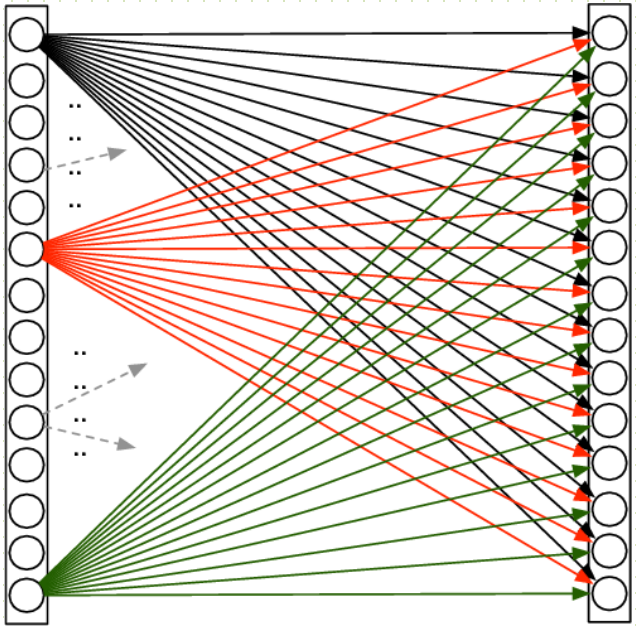


# Redes Neuronales Convolucionales

## Capa “Fully Connected”

- Es una capa que se encuentra completamente conectada (como en redes neuronales ordinarias)

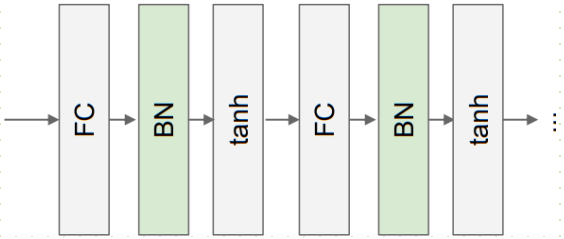


- Se le suele abreviar como FC

# Redes Neuronales Convolucionales

## Otras Capas

- Capas de normalización
  - *Batch Normalization* (BN)
    - Normaliza las activaciones (“salidas”) de las capas ocultas
    - Se coloca antes de la función de activación
    - Busca evitar el desplazamiento interno de la covarianza (“invariant covariant shift”)




  - Otras normalizaciones: layer normalization, group normalization, instance normalization

- Capas de regularización
  - *Dropout*: evita el sobre ajuste apagando aleatoriamente un porcentaje de neuronas
  - Modificaciones: DropConnect, SpatialDropout, Dropout adaptativo, variacional, etc.

# Redes Neuronales Convolucionales

## Aspectos Prácticos

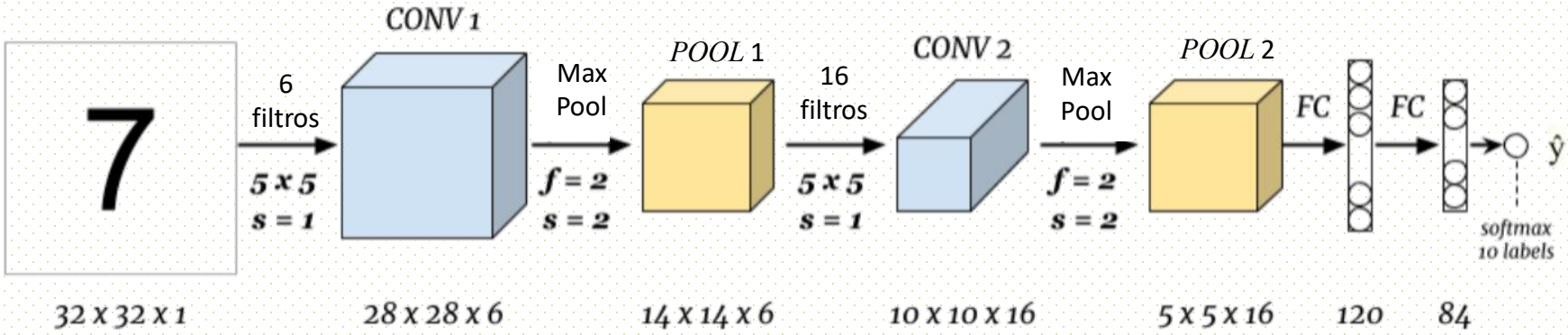
- Inicialización de parámetros (pesos)
  - Depende de la función de activación y puede ayudar en la convergencia
  - Algunas formas:
    - Inicialización aleatoria
    - Inicialización de Xavier
    - Inicialización de Kaiming/MSRA

- Regularización
  - Evita el sobreajuste ("overfitting") y permite un mejor entrenamiento
  - Algunas formas:
    - Aumento de datos ("Data augmentation")
    - Drop connect
    - Fractional pooling
    - Profundidad estocástica
    - Early stopping

# Redes Neuronales Convolucionales

## Arquitecturas de CNN

- Consiste en la disposición de capas (y sus características) en una CNN
- A medida que se va más adentro en la red:
  - El tamaño de la imagen tiende a decrecer
  - La profundidad tiende a incrementarse



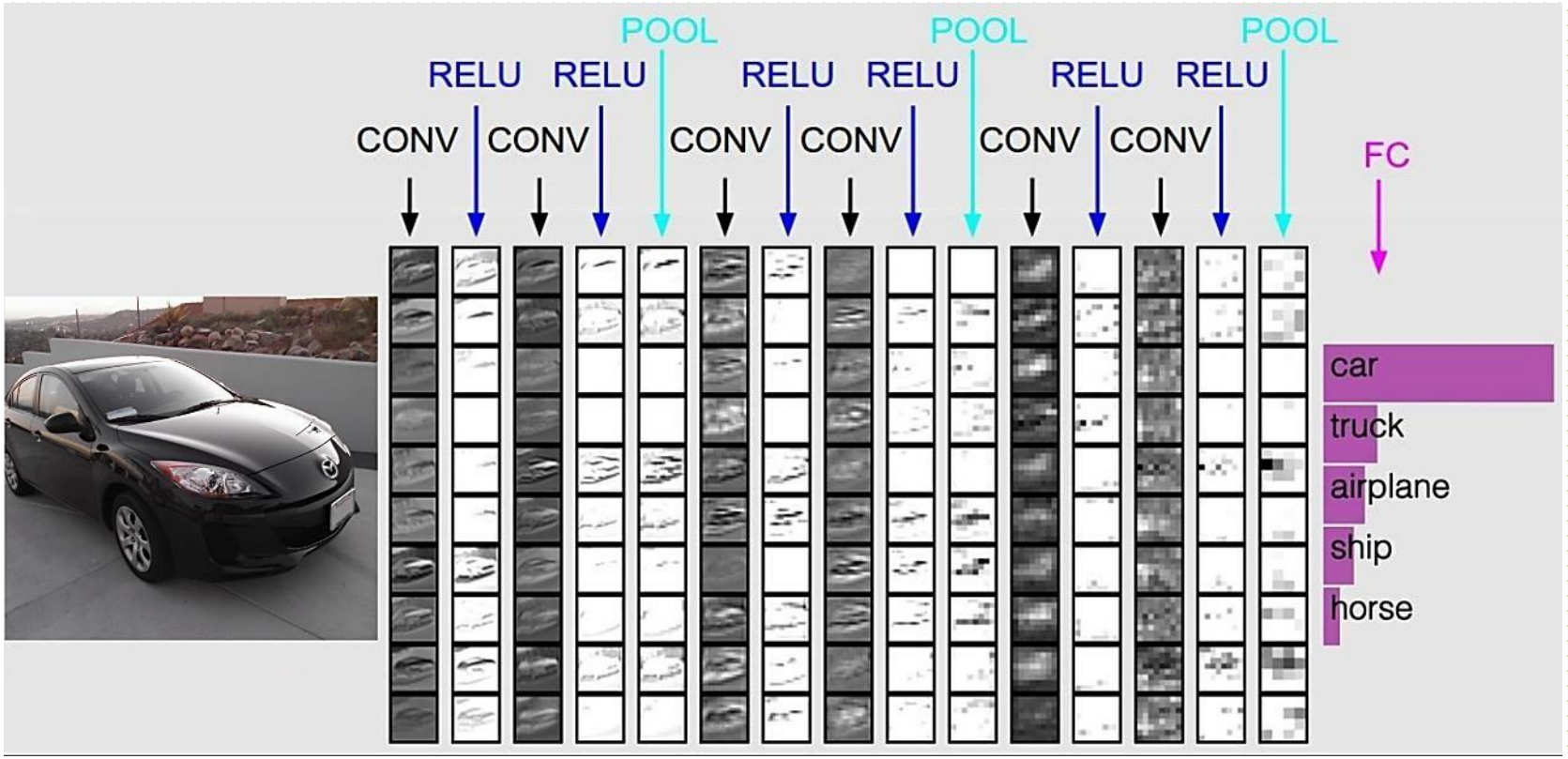

- Patrón común: Conv-Pool-Conv-Pool-FC-FC-FC-Softmax
- Existen diversas arquitecturas bastante conocidas

LeCun, Yann, et al. "Gradient-based learning applied to document recognition." (1998)

# Redes Neuronales Convolucionales

## Arquitecturas de CNN

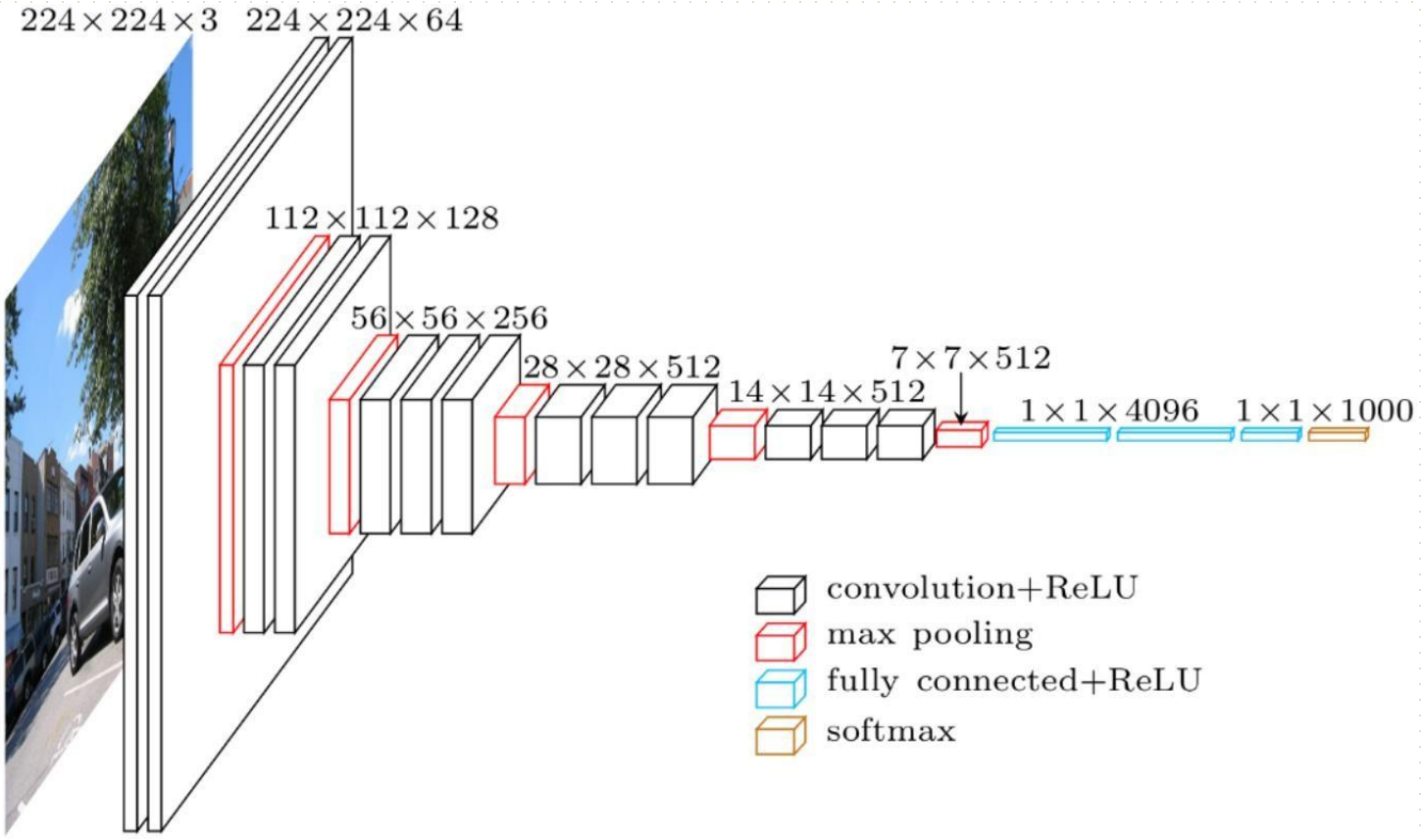
- Ejemplo de capas en una arquitectura de clasificación



# Redes Neuronales Convolucionales

## Arquitecturas de CNN

- VGG-16




K.Simonyan and A.Zisserman. "Very deep convolutional networks for large-scale image recognition." (2014).

# Redes Neuronales Convolucionales

## Arquitecturas de CNN

- VGG-16

Capas convolucionales:

(Conv1): Conv2D(64, (3, 3), activation='relu', padding='same', input_shape=(224, 224, 3))
(Conv2): Conv2D(64, (3, 3), activation='relu', padding='same')
(MaxPool1): MaxPooling2D((2, 2), strides=(2, 2))
(Conv3): Conv2D(128, (3, 3), activation='relu', padding='same')
(Conv4): Conv2D(128, (3, 3), activation='relu', padding='same')
(MaxPool2): MaxPooling2D((2, 2), strides=(2, 2))
(Conv5): Conv2D(256, (3, 3), activation='relu', padding='same')
(Conv6): Conv2D(256, (3, 3), activation='relu', padding='same')
(Conv7): Conv2D(256, (3, 3), activation='relu', padding='same')
(MaxPool3): MaxPooling2D((2, 2), strides=(2, 2))
(Conv8): Conv2D(512, (3, 3), activation='relu', padding='same')
(Conv9): Conv2D(512, (3, 3), activation='relu', padding='same')
(Conv10): Conv2D(512, (3, 3), activation='relu', padding='same')
(MaxPool4): MaxPooling2D((2, 2), strides=(2, 2))
(Conv11): Conv2D(512, (3, 3), activation='relu', padding='same')
(Conv12): Conv2D(512, (3, 3), activation='relu', padding='same')
(Conv13): Conv2D(512, (3, 3), activation='relu', padding='same')
(MaxPool5): MaxPooling2D((2, 2), strides=(2, 2))

Capas totalmente conectadas:

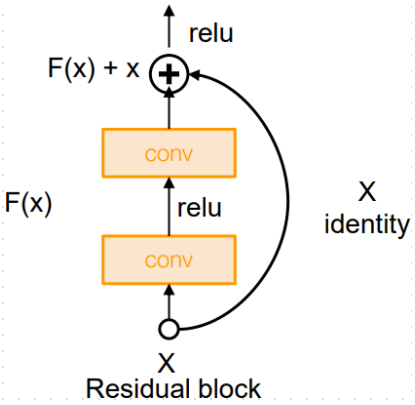
(Flatten): Flatten() (Dense1): Dense(4096, activation='relu')
(Dense2): Dense(4096, activation='relu')
(Output): Dense(1000, activation='softmax')



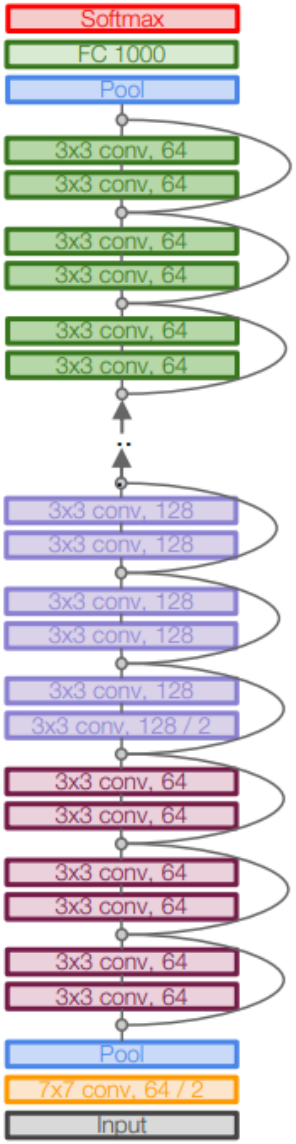
# Redes Neuronales Convolucionales

## Arquitecturas de CNN

- ResNet
  - Es una familia de arquitecturas para entrenar redes cada vez más profundas
  - El “bloque residual” permite tener redes más profundas




Al aumentar las capas no se degrada la red




- Variantes comunes: ResNet-18, ResNet-34, ResNet-50, ResNet-101, ResNet-152, ResNet-200 (versión más profunda, con 200 capas), ResNeXt, Wide ResNet, ResNetD, ResNetX,

# Redes Neuronales Convolucionales

## Arquitecturas de CNN

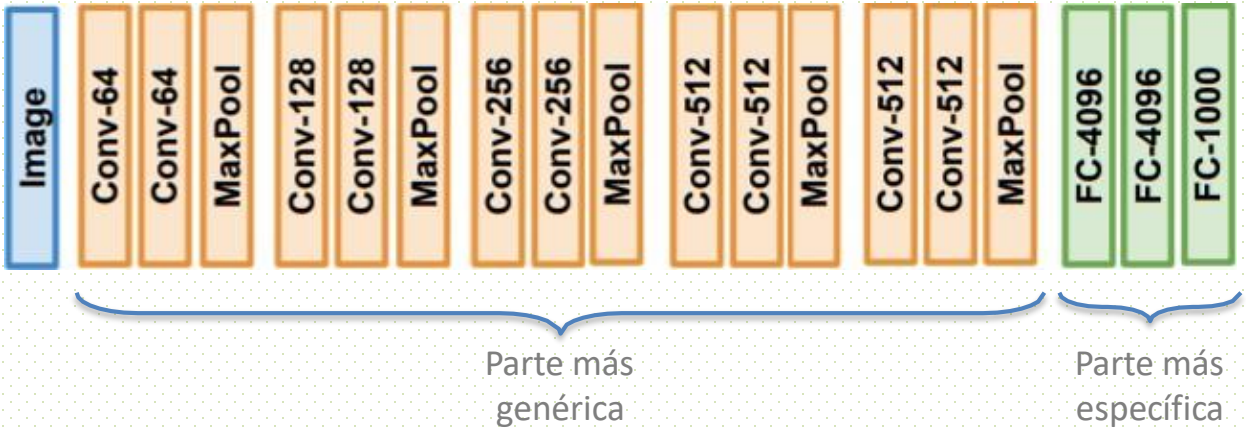
- Otras arquitecturas
  - Inception (GoogLeNet): utilizan convoluciones de diferente tamaño en paralelo
  - Arquitecturas enfocadas en eficiencia
    - *MobileNet*: para dispositivos móviles (convoluciones separables, factorización)
    - *SqueezeNet*: reduce el número de parámetros y operaciones
    - *ShuffleNet*: utiliza convolución de grupo y reorganiza los canales de entrada
  - Arquitecturas enfocadas en profundidad y precisión
    - *DenseNet*: concatenan las salidas de las capas anteriores
    - *Xception*: usan operadores de convolución separables en profundidad
    - *EfficientNet*: escalan la resolución, profundidad y ancho de manera compuesta
    - *NASNet*: usa refuerzo para encontrar la arquitectura “óptima”
    - *SENet*: introduce bloques “squeeze-and-excitation”
    - *CapsuleNet*: usa grupos de neuronas (cápsulas)

# Contenido

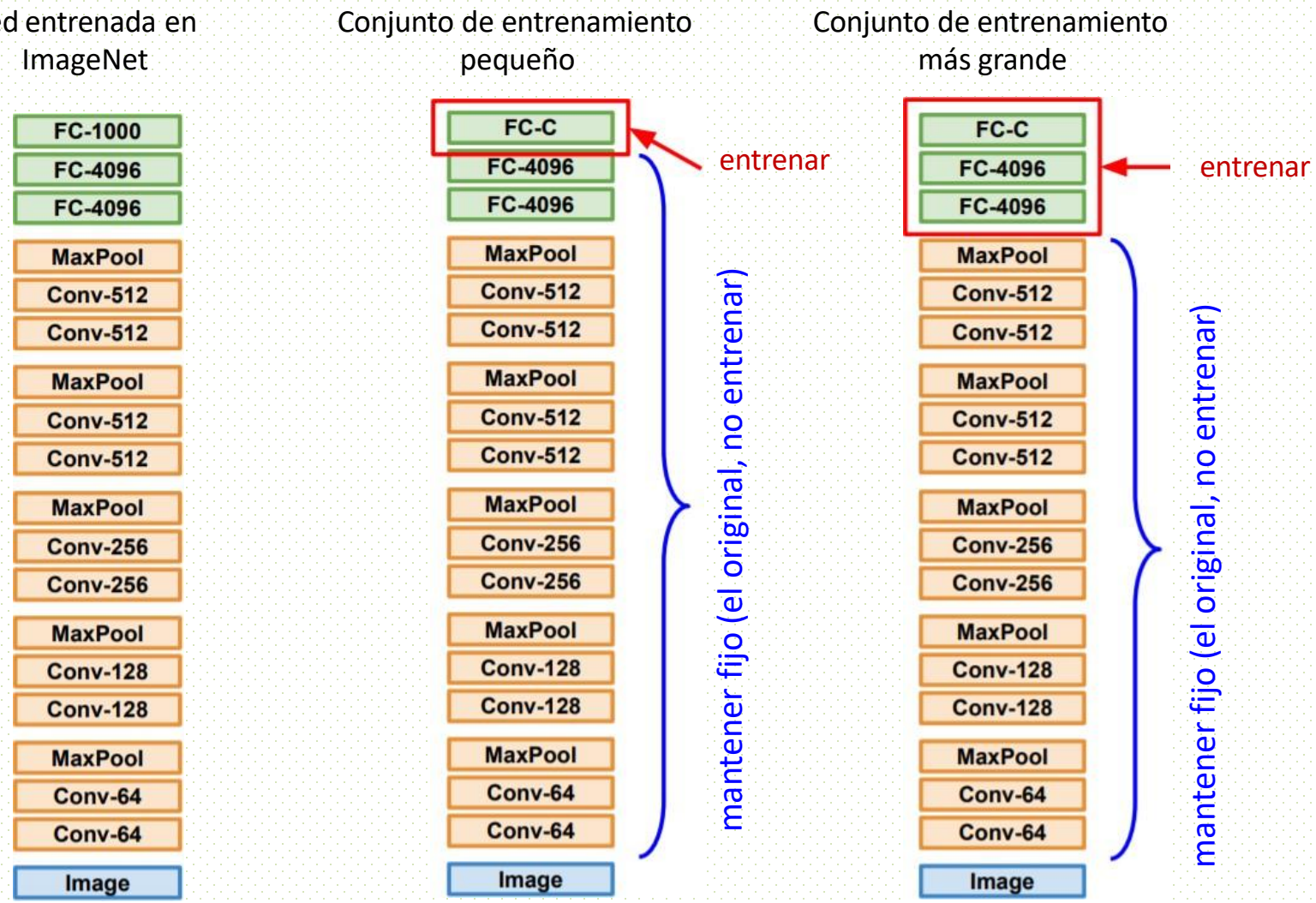
1. Redes Neuronales
2. Red Neuronales Convolucionales (CNN)
3. **Transferencia del Aprendizaje**
4. Redes Neuronales Recurrentes (RNN)

# Transferencia del Aprendizaje

- Problema en CNNs:
  - Se requiere muchos datos para entrenar/usar CNNs
  - Requiere mucho tiempo de entrenamiento
- Alternativa
  - Aprovechar parte de alguna red ya entrenada (es común en la práctica)
  - Se conoce como “Transfer Learning”
- Idea



# Transferencia del Aprendizaje



# Transferencia del Aprendizaje

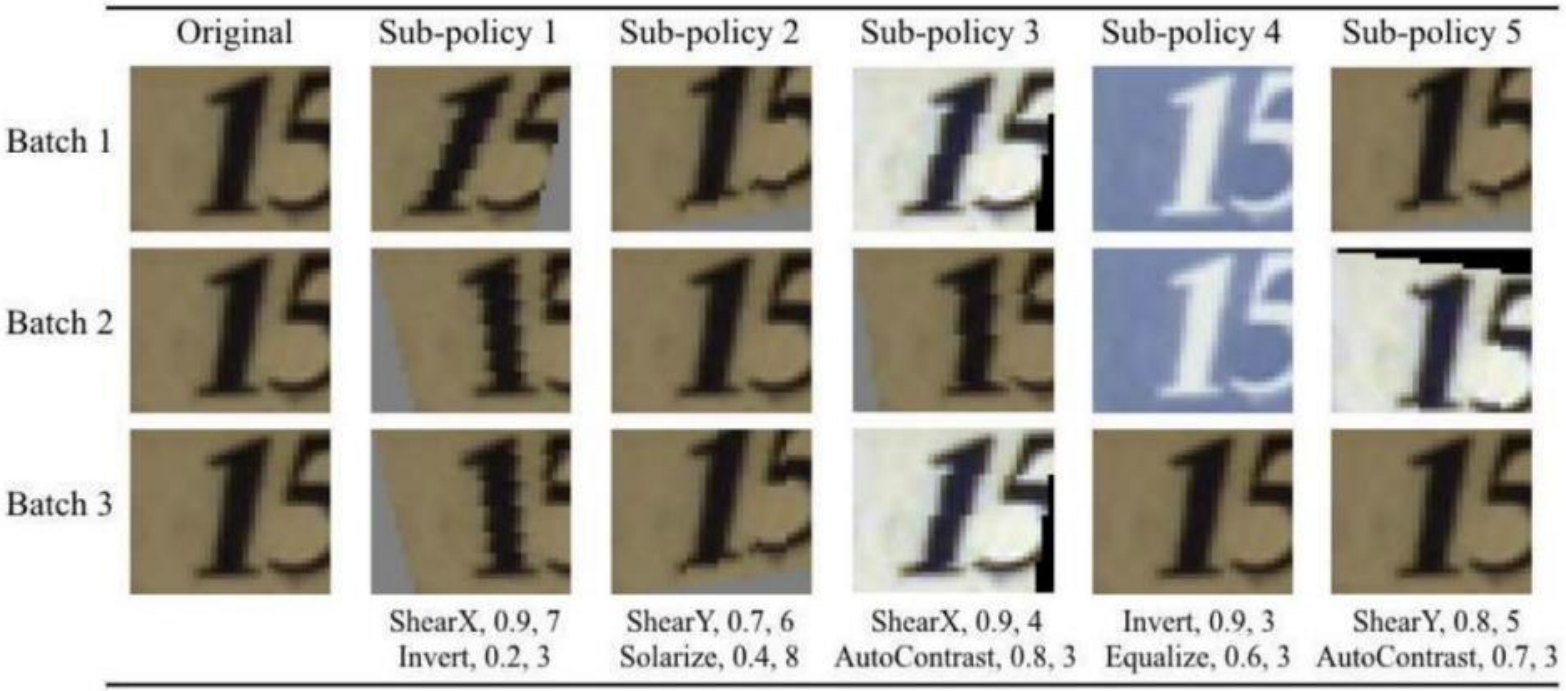
- Sugerencias para utilizar transfer learning:
  - Según la cantidad de datos que se tenga
  - Según la similaridad o diferencia con el sistema pre-entrenado

| | **Dataset muy similar** | **Dataset muy diferente** |
|---|---|---|
| **Muy pocos datos** | Hacer un “finetune” del clasificador de la capa final | Intentar “data augmentation” o recolectar más datos |
| **Bastantes datos** | Hacer un “finetune” de algunas capas | Hacer un “finetune” de un número más grande de capas |

- Existen varias redes pre-entrenadas en grandes conjuntos de datos:
  - Se sugiere aprovecharlas cuando sea posible
  - Para transfer learning se puede usar “Data Augmentation”

# Transferencia del Aprendizaje

- Data Augmentation
  - Incrementar los datos (imágenes) mediante operaciones como: traslación, rotación, escalado, shearing, distorsión de lentes, deformaciones, variaciones en intensidad, etc.
  - Ejemplo:

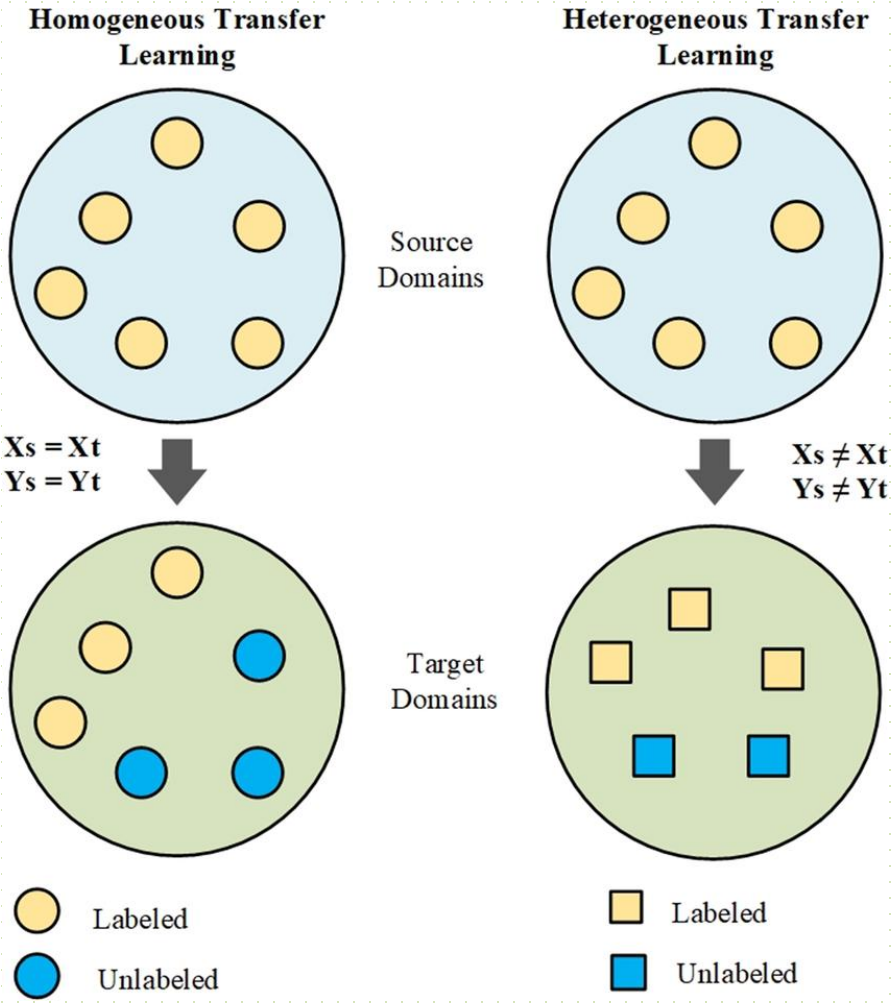



Cubuk et al., "AutoAugment: Learning Augmentation Strategies from Data", CVPR 2019

# Transferencia del Aprendizaje

***Homogéneo:*** Transfieres conocimiento entre tareas donde los datos de origen (fuente) y los de destino (objetivo) comparten el mismo espacio de características. Es decir, "hablan el mismo idioma".

***Heterogéneo***: Transfieres conocimiento entre tareas donde los datos de origen y destino tienen espacios de características diferentes. Es decir, "hablan idiomas distintos" y necesitas un traductor.

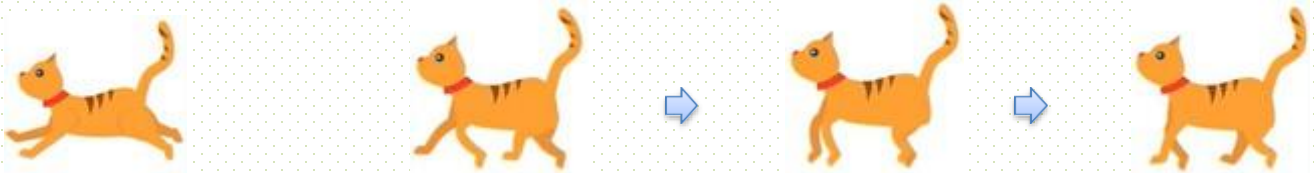


# Contenido

1. Redes Neuronales
2. Redes Neuronales Convolucionales (CNN)
3. Transferencia del Aprendizaje
4. **Redes Neuronales Recurrentes (RNN)**

# Redes Neuronales Recurrentes

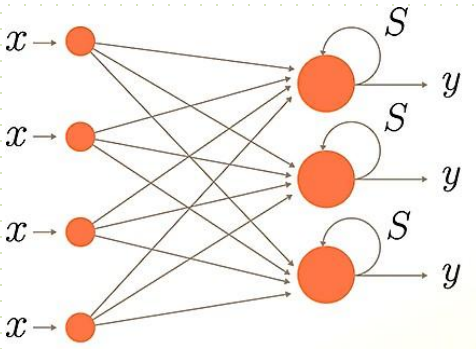
- Muchas aplicaciones involucran secuencias: existen “Dependencias Temporales”
  - La salida actual depende de la entrada actual y de las entradas pasadas (video, reconocimiento del habla)
  - Debe haber “memoria”



¿Está saltando?

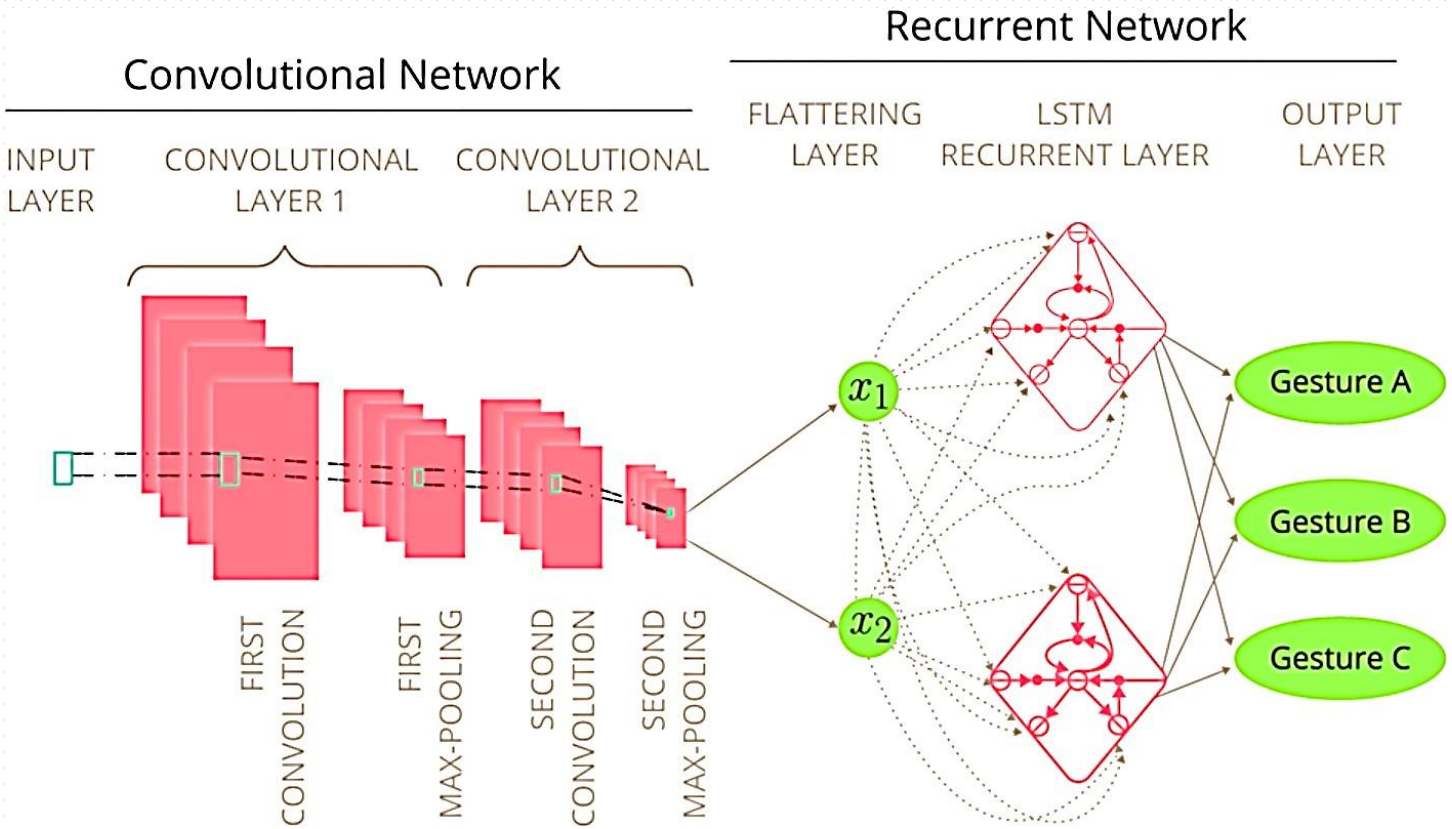
¿Está caminando?

- RNN (Recurrent Neural Networks)
  - Recurrente: ocurre a menudo o repetidamente
  - Se realiza la misma tarea para cada elemento de la (secuencia) de entrada
  - Trabajan con secuencias



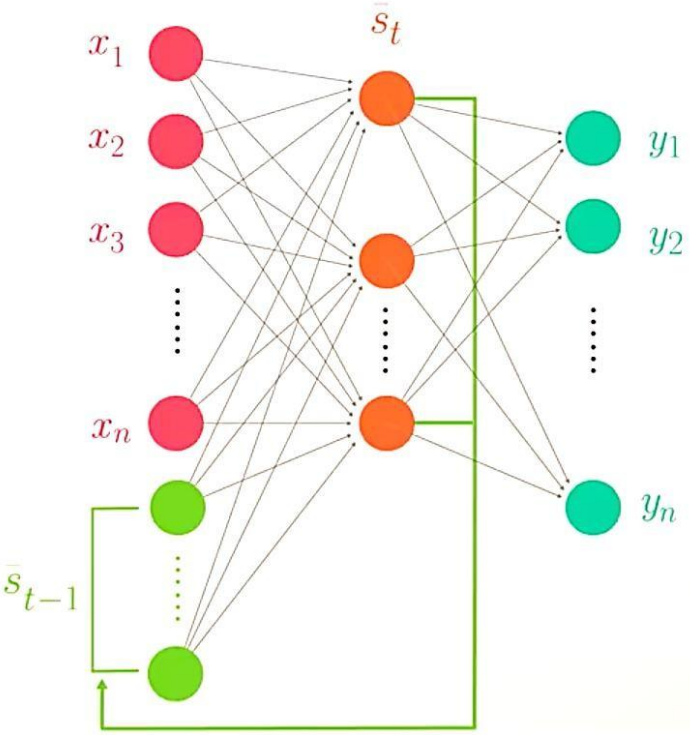
# Redes Neuronales Recurrentes

- Reconocimiento de gestos



# Redes Neuronales Recurrentes

- RNN simple
  - Las salidas de las capas ocultas se comportan como entradas a las capas (“estados”)



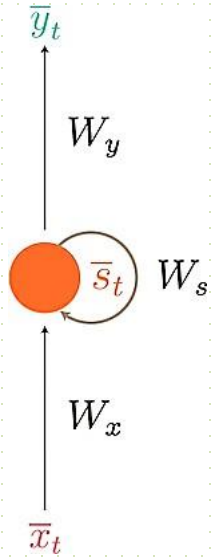

RNN simple de Elman




Analogía con una máquina de estados finitos

# Redes Neuronales Recurrentes

- Esquema de RNN
  - La salida depende de la entrada actual y del estado
  - El estado (S) contiene información del “pasado”

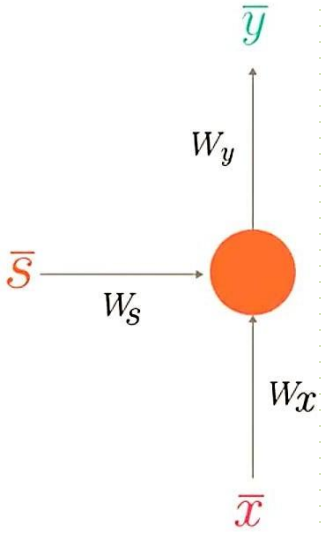



“folded representation”

$$y_t = W_y s_t$$

$$s_t = g(W_x x_t + W_s s_{t-1})$$

$x_t$: entrada en tiempo $t$
$y_t$: salida en tiempo $t$
$s_t$: estado en tiempo $t$

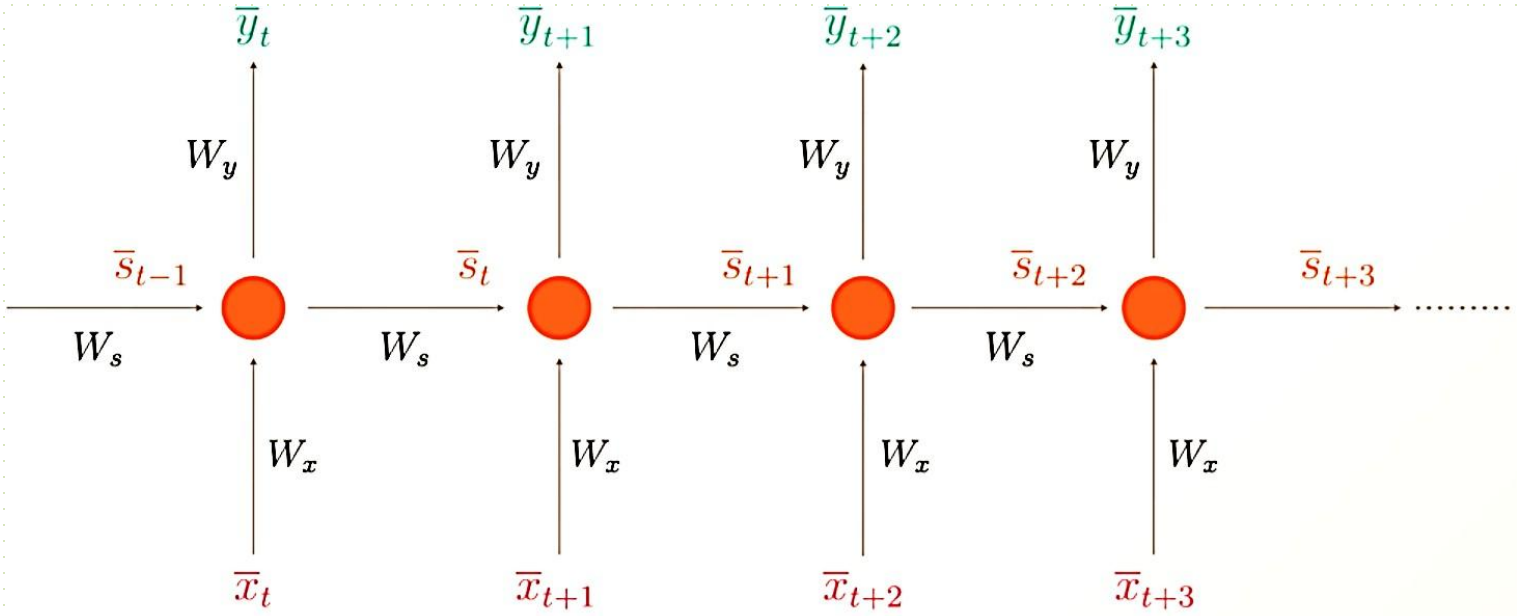



“unfolded representation”

$W_x$: matriz que relaciona las entradas con el estado
$W_y$: matriz que relaciona el estado con la salida
$W_s$: Matriz que relaciona el estado anterior con el actual

# Redes Neuronales Recurrentes

- Esquema de RNN
  - Ejemplo con varias entradas (secuenciales)



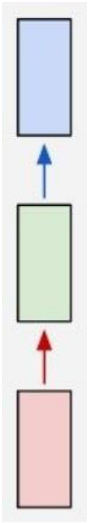

  - Es solo una representación temporal
    - Los pesos (en cada etapa) son los mismos

# Redes Neuronales Recurrentes

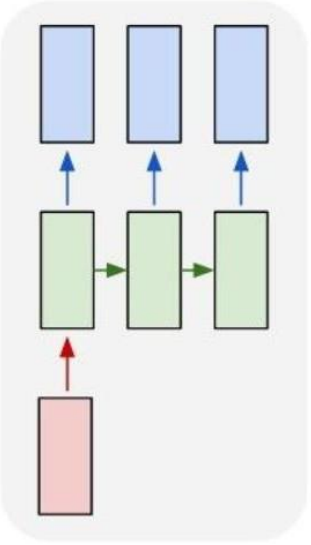
## Arquitecturas

- One to one
  - Entrada fija (única): imagen, vector
  - Salida única: clasificación
- One to many:
  - Entrada fija (única): imagen, vector
  - Salida variable (secuencia): texto, descripción
  - Ejemplo: etiquetado de imagen

One to one



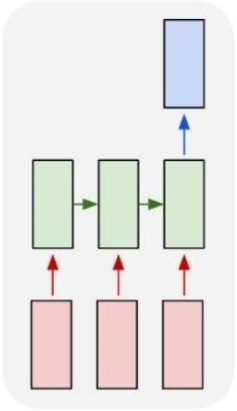
One to many



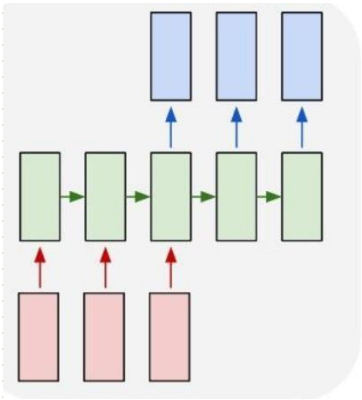
# Redes Neuronales Recurrentes

## Arquitecturas

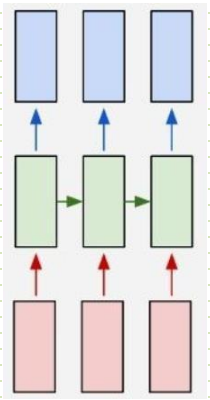
- Many to one
  - Entrada variable (secuencia): texto, video
  - Salida fija: clasificación, sentimiento, tipo de actividad
  - Ejemplo: clasificación del sentimiento en texto
- Many to many
  - Entrada variable (secuencia): texto, video
  - Salida variable (secuencia): texto, decisión (para cada elemento de la secuencia)
  - Ejemplo: traducción (secuencia de palabras en idioma A al idioma B)



Many to one



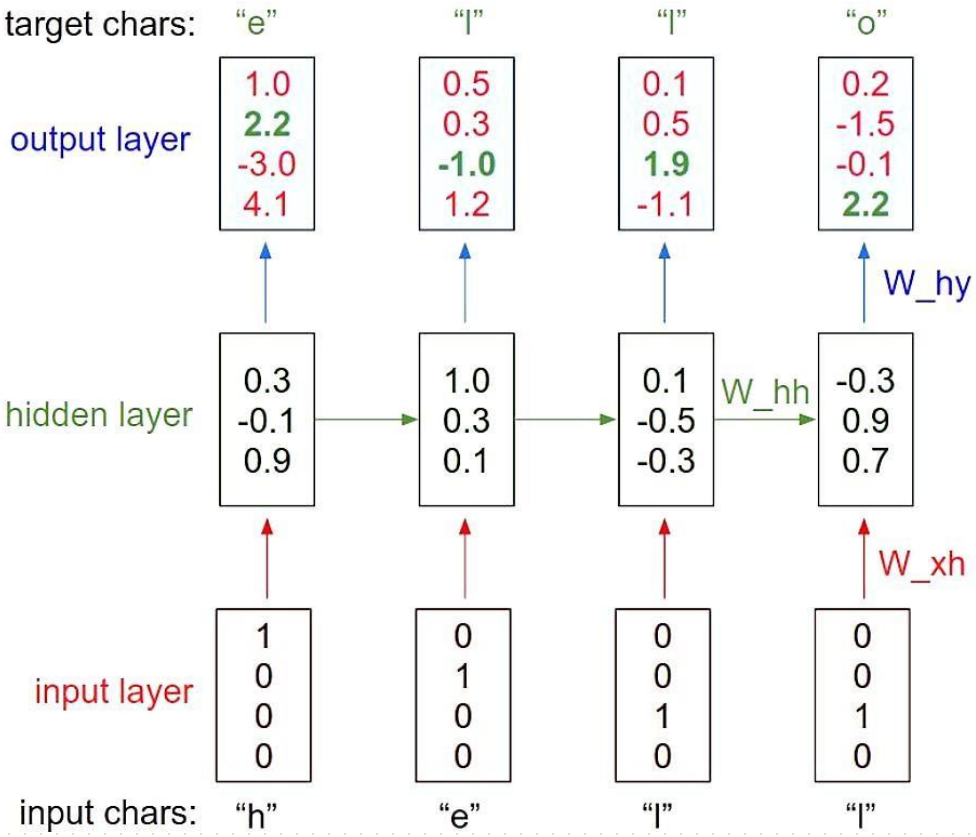
Many to many



# Redes Neuronales Recurrentes

## Ejemplo

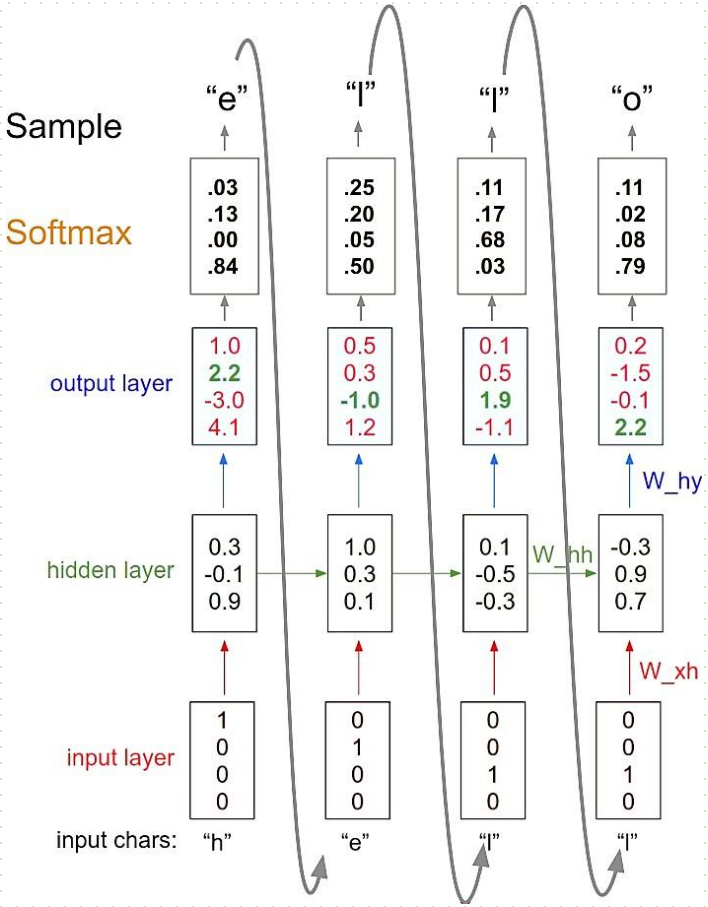
- Dadas unas letras, predecir las siguientes letras
  - Vocabulario: {h, e, l, o}
  - Ejemplo de secuencia de entrenamiento: “hello”



# Redes Neuronales Recurrentes

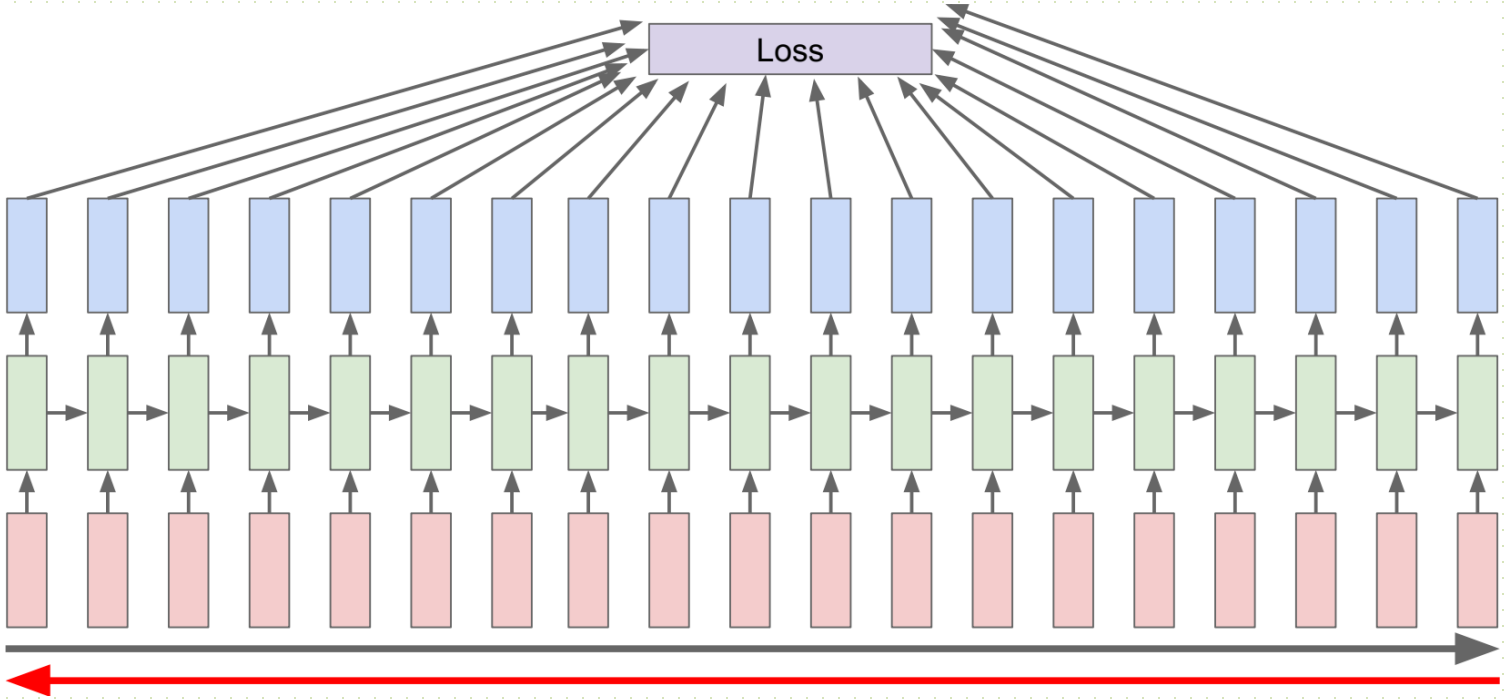
## Ejemplo

- Dadas unas letras, predecir las siguientes letras
  - En la prueba



# Redes Neuronales Recurrentes

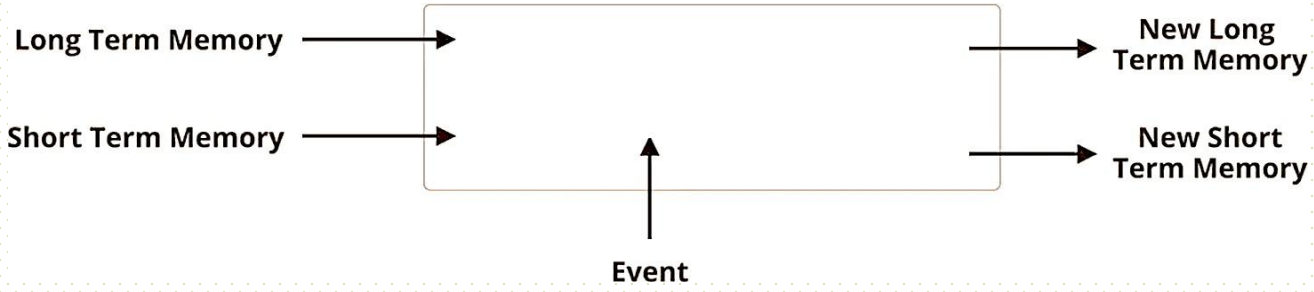
- BPTT = BackPropagation Through Time
- Función de costo (de manera general): suma de costo de cada etapa




- Problemático si secuencia es muy grande (memoria, tiempo)

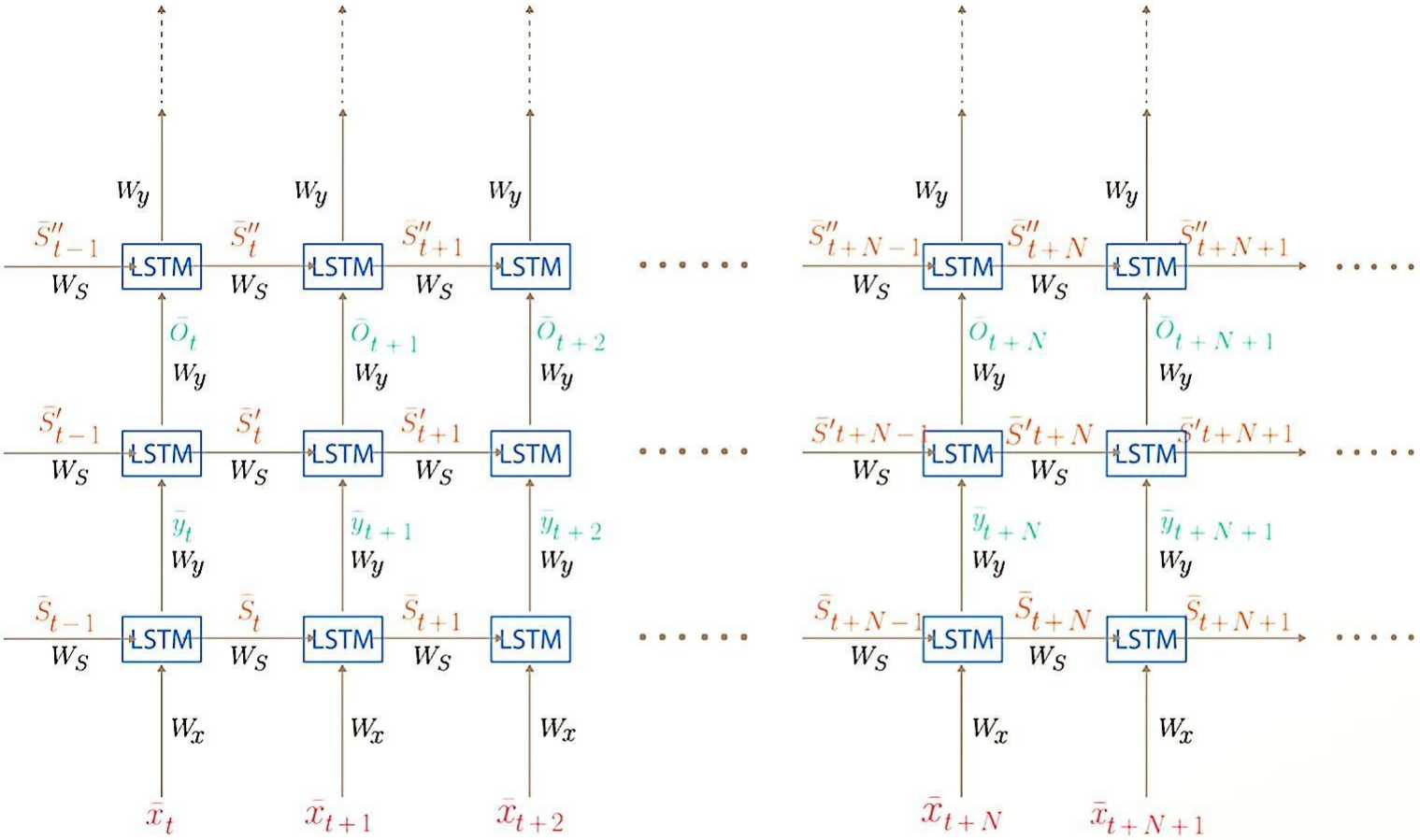
# Redes Neuronales Recurrentes: LSTM

- LSTM = Long-short Term Memory
  - Evita la pérdida de información en el tiempo
  - Evita la desaparición de los gradientes (vanishing gradients)
- Entradas y salidas (E/S) de un LSTM
  - E/S de largo plazo (“long term memory”): LTM
  - E/S de corto plazo (“short term memory”): STM
  - Entrada actual (“evento”)



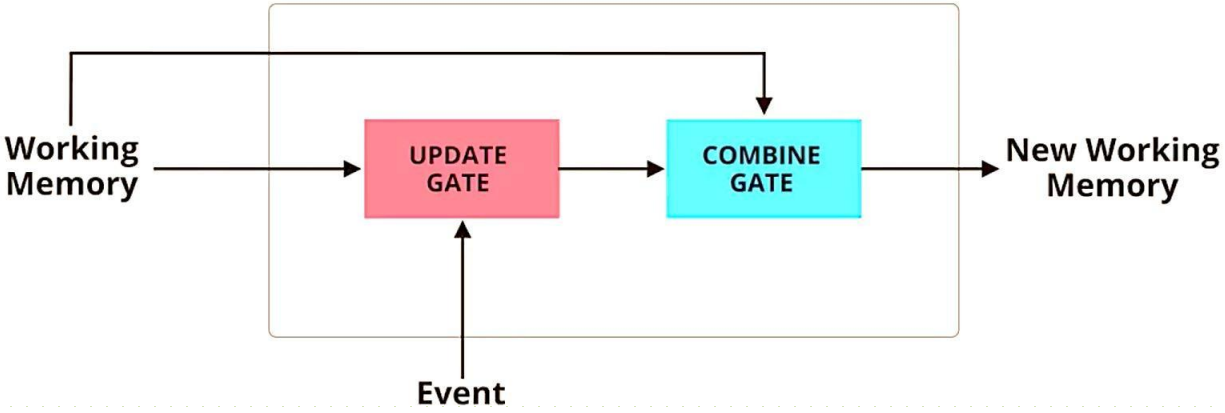
# Redes Neuronales Recurrentes: LSTM

- Se puede formar varias capas de LSTM



# Redes Neuronales Recurrentes: GRU

- GRU = Gated Recurrent Unit
- Se introdujo en 2014 para resolver el problema de los gradientes que desaparecen (“vanishing gradients”)
- Se puede considerar una variación de LSTM




- Es computacionalmente más eficiente que LSTM (su estructura es más simple)
  - De manera práctica tienden a funcionar mejor que LSTM (excepto cuando se requiere relaciones de muy largo plazo)

# Referencias


- R. Szeliski. *Computer Vision: Algorithms and Applications*. Springer, 2010 (*Capítulos* 4, 14)

- D. Forsyth, J. Ponce: Computer Vision, A Modern Approach, Pearson Education, NJ 2012 (*Capítulos* 5, 16, 17)

- R. Siegwart, R.I. Nourbakhsh, and D. Scaramuzza. *Introduction to Autonomous Mobile Robots*. MIT press, 2011. (*Capítulo 4*)